

CI/CD Pipeline

Complete Guide — Concepts, Flow, Usage,
and Applying It to the Hungry_JU Project

Practical reference for Continuous Integration & Continuous Delivery
Includes ready-to-use GitHub Actions workflows

Target project: **Hungry_JU** — Next.js 16 (App Router), React 19, Tailwind 4
Repository: github.com/HumairaSadiah/Hungry_JU

Contents

Part I — Understanding CI/CD

1. What CI/CD Is and Why It Exists
2. Core Vocabulary
3. Anatomy of a Pipeline — The Full Flow
4. Stage-by-Stage Breakdown
5. Triggers and Branching Strategy
6. Environments and Promotion
7. Deployment Strategies
8. Secrets and Security in the Pipeline
9. Caching, Speed, and Cost
10. Tool Comparison

Part II — Using a Pipeline Day to Day

11. The Developer Workflow
12. Reading a Failed Pipeline
13. Quality Gates and Branch Protection
14. Measuring the Pipeline (DORA Metrics)

Part III — Applying CI/CD to Hungry_JU

15. Current State and Gaps
16. Step 1 — Prepare the Repository
17. Step 2 — The CI Workflow (lint, format, build)
18. Step 3 — Adding Tests to the Pipeline
19. Step 4 — Branch Protection Rules
20. Step 5 — The CD Workflow (deploy)
21. Step 6 — Database Migrations in CD
22. Step 7 — Secrets and Environment Variables
23. Step 8 — Extras (security scan, Docker, previews)
24. Rollout Plan and Checklist
25. Troubleshooting Reference

Part I — Understanding CI/CD

1. What CI/CD Is and Why It Exists

CI/CD is the practice of automating everything that happens between "a developer writes code" and "that code runs in production". A machine — not a person — checks, builds, tests, and ships the code, every time, in the same order.

The three terms

Term	Meaning
CI Continuous Integration	Every commit is automatically merged into a shared branch and verified: install dependencies, lint, type-check, build, run tests. Goal: the main branch is always in a working state, and integration bugs surface in minutes, not weeks.
CD Continuous Delivery	Every verified commit is automatically packaged into a deployable artifact and pushed to a staging environment. A human presses the final button for production.
CD Continuous Deployment	Same as delivery, but with no human button. Passing the pipeline <i>is</i> the release decision. Requires strong test coverage and fast rollback.

The problem it solves

Without automation, every release depends on someone remembering the right steps in the right order on the right machine. That produces the classic failure modes:

- **"Works on my machine."** The build depends on a local Node version, a local `.env`, an uncommitted file. CI builds in a clean container, so hidden dependencies fail immediately.
- **Integration hell.** Five people branch off for two weeks; merging is a multi-day fight. CI forces small, frequent merges.
- **Broken main.** Someone pushes code that does not compile; everyone else is blocked. A required CI check makes that push impossible.
- **Manual deploy mistakes.** Wrong branch, forgotten migration, skipped build step. A scripted deploy does the same thing every time.
- **Slow feedback.** A bug found three weeks later costs far more than one found in three minutes.

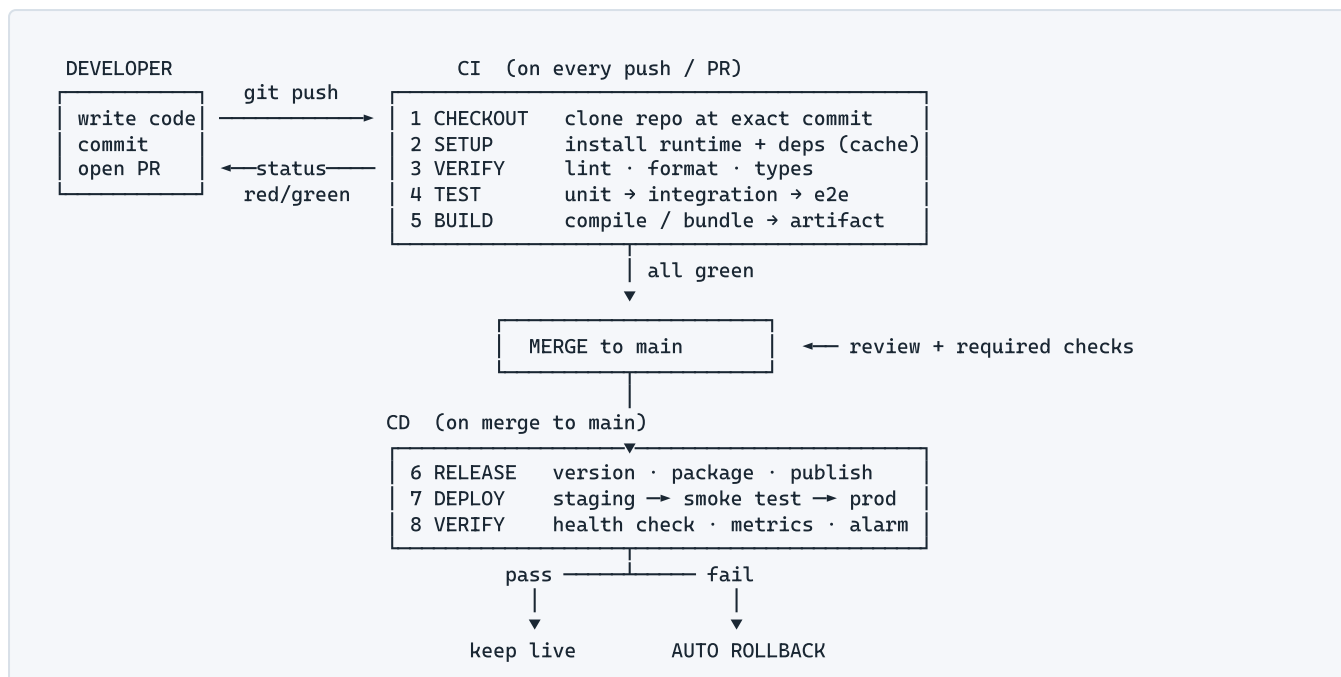
Core principle: if a step must be done correctly every time, it should be a script in the repository, not an instruction in someone's head. The pipeline is that script, executed by a machine, with the log kept as evidence.

2. Core Vocabulary

Term	Definition
Pipeline / Workflow	The whole automated sequence triggered by an event. In GitHub Actions, one YAML file under <code>.github/workflows/</code> .
Trigger / Event	What starts the pipeline: a push, a pull request, a tag, a schedule, a manual click, an API call.
Job	A group of steps that runs on one machine. Jobs run in parallel by default; <code>needs:</code> makes one wait for another.
Step	One unit inside a job — either a shell command (<code>run:</code>) or a reusable action (<code>uses:</code>).
Runner / Agent	The machine executing the job. GitHub-hosted (fresh Ubuntu VM every run) or self-hosted (your own server).
Artifact	A file produced by a job and kept for later jobs or for download — a build output, a coverage report, a test log, a Docker image.
Cache	Reused data across runs to save time — typically <code>node_modules</code> or the package-manager store. Never a correctness dependency.
Matrix	Running the same job many times with different inputs (Node 20 and Node 22, Linux and Windows).
Environment	A named deploy target (<code>staging</code> , <code>production</code>) holding its own secrets and optional approval rules.
Gate / Required check	A rule that blocks merging or deploying until a specific job passes.
Rollback	Returning production to the previous known-good release.
Idempotent	Running the step twice produces the same result as running it once — required for safe retries.

3. Anatomy of a Pipeline — The Full Flow

Almost every real pipeline is the same seven phases. Tools differ; the shape does not.



Two loops, not one

- **The fast loop (CI)** runs on every push and pull request. It must finish in under ~10 minutes or people start ignoring it. Its only job is to answer: *is this change safe to merge?*
- **The slow loop (CD)** runs after merge. It may take longer, involve approvals, migrations, and staged rollout. Its job: *get this change to users without breaking them.*

Ordering rule: cheap and fast checks first, expensive ones last. Lint fails in 20 seconds; end-to-end tests take 8 minutes. Running lint first means a missing semicolon never costs 8 minutes of runner time. This is called *fail fast*.

4. Stage-by-Stage Breakdown

4.1 Checkout

The runner starts empty. It clones the repository at the exact commit SHA that triggered the run — not "latest main". This is what makes a run reproducible: the same SHA always builds the same thing.

4.2 Setup / Install

Install the language runtime at a pinned version and install dependencies from the lockfile:

- Use `npm ci`, never `npm install`, in CI. `ci` deletes `node_modules`, installs exactly what `package-lock.json` says, and fails if the lockfile and `package.json` disagree. `install` may silently upgrade packages and rewrite the lockfile — that makes builds non-reproducible.
- Pin the runtime version (`node-version: 22`), so the pipeline is not silently upgraded under you.

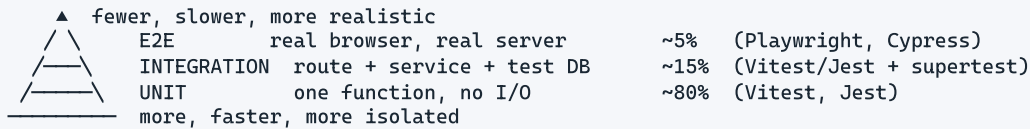
4.3 Verify — static checks

No code is executed; the source is analysed.

- **Lint** (ESLint) — bug patterns, unused variables, bad imports.

- **Format check** (Prettier `--check`) — fails if files are not formatted. Never let CI auto-format and push; it creates commit loops. It reports; the developer fixes.
- **Type check** (`tsc --noEmit`) — for TypeScript projects.

4.4 Test — the pyramid



- **Unit tests** — pure logic, no database, no network. Milliseconds each. Run on every push.
- **Integration tests** — several layers together against a throwaway database (a service container in CI). Seconds each.
- **E2E tests** — a real browser driving the running app. Minutes. Run on PRs to main and before deploy, not on every commit.

Flaky tests are worse than no tests. A test that fails randomly teaches the team to re-run the pipeline instead of reading it. Quarantine flaky tests immediately, then fix or delete them.

4.5 Build

Produce the deployable artifact — a bundle, a container image, a binary. Two rules:

- **Build once, deploy many.** The exact artifact tested in staging is the artifact promoted to production. Rebuilding per environment means production runs code that was never tested.
- **Configuration comes from the environment, not the build.** Secrets and URLs are injected at deploy time. (Caveat for front-end frameworks: values inlined into the client bundle at build time — anything `NEXT_PUBLIC_*` — are baked in, so those must be correct at build time and must never contain secrets.)

4.6 Release / Package

Tag a version, generate a changelog, push the image to a registry, publish the package. The artifact gets an immutable identity (a semver tag or the commit SHA) so any deploy can be traced back to exact source.

4.7 Deploy and Verify

Ship the artifact, then prove it works: a health-check endpoint, a smoke test hitting a few critical paths, error-rate and latency monitoring for a few minutes. If verification fails, roll back automatically. **A deploy with no verification step is not automation, it is just a faster way to break production.**

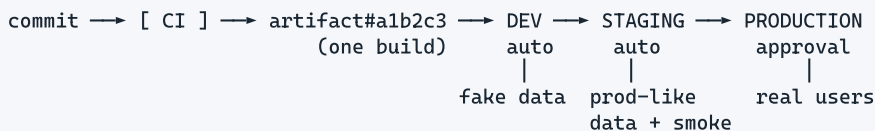
5. Triggers and Branching Strategy

Trigger	Typically runs
<code>push</code> to feature branch	Lint, unit tests, build. Fast feedback for the author.
<code>pull_request</code> to main	Full CI: lint, format, all tests, build, security scan. These are the required checks.
<code>push</code> to main (post-merge)	CI again, then deploy to staging, then production.
<code>tag v*</code>	Cut a release: build, publish, deploy production.
<code>schedule</code> (cron)	Nightly E2E suite, dependency audit, stale-cache cleanup.
<code>workflow_dispatch</code>	Manual run with inputs — deploys, rollbacks, one-off jobs.

Branching models

- **Trunk-based (recommended for small teams).** Short-lived branches off `main`, merged within a day or two behind feature flags if incomplete. Simplest pipeline: PR → CI, merge → deploy.
- **GitHub Flow.** Trunk-based plus a deploy-from-PR preview environment. Good default for web apps.
- **GitFlow** (`develop`, `release/*`, `hotfix/*`). Heavy; suits versioned/on-prem software with long release cycles. Overkill for a continuously deployed web app.

6. Environments and Promotion



Environment	Purpose	Data
Local	Development	Seeded fake data
Preview / PR	One disposable deployment per pull request, so reviewers click the change instead of imagining it	Shared dev DB or per-PR branch DB
Staging	Final rehearsal — same infra shape, same migrations, same config keys as production	Anonymised production-like
Production	Live users	Real

Promotion means moving the *same* artifact forward through these environments. Only the configuration changes.

7. Deployment Strategies

Strategy	How it works	Trade-off
Recreate	Stop old version, start new one	Simple; causes downtime. Fine for internal tools.
Rolling	Replace instances a few at a time	No downtime; two versions live at once, so APIs and DB schema must be backward compatible.
Blue-Green	Two identical stacks; deploy to idle one, test it, flip the router	Instant rollback (flip back); costs double infrastructure during the switch.
Canary	Route 1% → 10% → 50% → 100% of traffic to the new version, watching error rates at each step	Smallest blast radius; needs good metrics and traffic splitting.
Feature flags	Ship the code disabled, enable it for users separately from deploying	Decouples release from deploy — the most valuable one to adopt. Costs flag hygiene: dead flags must be deleted.

8. Secrets and Security in the Pipeline

- **Never commit secrets.** Store them in the CI provider's secret store (GitHub → Settings → Secrets and variables → Actions), or a vault. Commit only `.env.example` with empty values.
- **Scope secrets per environment.** The staging job must not be able to read the production database password.
- **Secrets are masked in logs**, but a secret you `echo` into a file, a URL, or an error message can still leak. Do not print them.
- **Pin third-party actions.** `uses: actions/checkout@v5` is a moving tag; a full commit SHA is immutable and cannot be swapped under you.
- **Least privilege for the pipeline token.** Declare `permissions:` explicitly and default to `contents: read`.
- **Never run untrusted PR code with secrets.** On public repos, `pull_request` from a fork runs without secrets by design. Do not "fix" that with `pull_request_target` unless you fully understand it — it is a common repository-takeover vector.
- **Automated scanning in the pipeline:** dependency audit (`npm audit` , Dependabot), secret scanning (gitleaks), static analysis (CodeQL), container scanning (Trivy).

9. Caching, Speed, and Cost

- **Cache the dependency store**, keyed on the lockfile hash. New lockfile → new cache key → clean install.
- **Cache the framework build cache** (for Next.js: `.next/cache`) keyed on lockfile + source hash. Cuts build time substantially.
- **Parallelise independent jobs.** Lint and unit tests have no reason to run sequentially.
- **Cancel superseded runs.** Use a `concurrency` group so a new push cancels the previous in-flight run on the same branch.
- **Only run what changed** in a monorepo — path filters or a task runner with a dependency graph.

- **Budget:** target under 10 minutes for PR CI. Beyond ~15 minutes people context-switch and the feedback loop dies.

10. Tool Comparison

Tool	Best for	Notes
GitHub Actions	Anything already on GitHub	Zero setup, huge marketplace, generous free tier for public repos. Recommended for Hungry_JU.
GitLab CI	GitLab-hosted repos	Single <code>.gitlab-ci.yml</code> , strong built-in environments and registry.
Jenkins	Self-hosted, heavily customised, legacy estates	Most flexible, most maintenance. You run the server and plugins.
CircleCI / Travis	SaaS CI independent of host	Good caching and parallelism; another vendor to manage.
Vercel / Netlify	Front-end deploy (CD only)	Git-connected: push → preview URL → production. Pairs with GitHub Actions for the CI half.
Argo CD / Flux	Kubernetes GitOps	Cluster state is a Git repo; the operator reconciles continuously.

Part II — Using a Pipeline Day to Day

11. The Developer Workflow

1. **Branch.** `git switch -c feat/order-cancel` off up-to-date `main`.
2. **Run the same checks locally first.** The pipeline should hold no surprises — `npm run lint && npm run format:check && npm run build`. Anything CI runs, you can run.
3. **Commit small, push often.** Each push triggers the fast loop.
4. **Open a pull request.** Full CI runs and reports back on the PR as check marks.
5. **Red check → fix and push again.** Do not merge past a failure; do not disable the check.
6. **Review + green checks → merge.** Branch protection enforces both.
7. **Merge triggers CD.** Deploy to staging, smoke test, then production (auto or approved).
8. **Watch the release.** Health checks and error rate for a few minutes. Roll back if it degrades — rolling back is a normal, blameless action, not a failure.

Golden rule: the pipeline is the single source of truth about whether code works. If it is green but production breaks, the fix is to *add a check to the pipeline*, not to add a step to a wiki page.

12. Reading a Failed Pipeline

1. **Which job failed?** Lint, test, build, and deploy failures mean completely different things.
2. **Open the failing step and read the first error**, not the last. Later errors are usually consequences.
3. **Reproduce locally** with the same command the workflow ran. It is printed verbatim in the log.
4. **If it passes locally but fails in CI**, the difference is environment: Node version, missing env var, case-sensitive filesystem (Linux runner vs Windows dev machine — `import './Button'` vs `./button` is a classic), timezone, or an uncommitted file.
5. **If it fails intermittently**, it is flaky: a race condition, a shared test database, a hard-coded port, or a time-dependent assertion.
6. **Re-running is diagnosis, not a fix.** Re-run once to confirm flakiness, then open an issue.

13. Quality Gates and Branch Protection

A pipeline nobody is required to pass is decoration. Enforce it on GitHub under **Settings → Branches → Add branch ruleset** for `main`:

- Require a pull request before merging (block direct pushes to `main`).
- Require status checks to pass — select the CI job names by name.
- Require branches to be up to date before merging (catches semantic conflicts).
- Require at least one approving review.
- Require conversation resolution.
- Block force pushes and deletion of `main`.

Optional stricter gates: minimum coverage threshold, no high-severity vulnerabilities, signed commits, linear history.

14. Measuring the Pipeline (DORA Metrics)

Metric	Question	Good
Deployment frequency	How often do you ship?	On demand / daily
Lead time for change	Commit → running in production?	< 1 day
Change failure rate	% of deploys causing an incident	< 15%
Mean time to restore	How fast can you recover?	< 1 hour

Also worth tracking: pipeline duration (p50/p95), flaky-test rate, and cache hit rate.

Part III — Applying CI/CD to Hungry_JU

15. Current State and Gaps

Observed in the repository:

Item	Status
Host	GitHub — <code>github.com/HumairaSadia/Hungry_JU</code> . GitHub Actions is the natural choice.
Stack	Next.js 16 (App Router), React 19, Tailwind 4, JavaScript (no TypeScript), Zod, bcryptjs, jsonwebtoken.
Existing scripts	<code>dev</code> , <code>build</code> , <code>start</code> , <code>lint</code> , <code>lint:fix</code> , <code>format</code> , <code>format:check</code> — <code>lint</code> and <code>format:check</code> are already CI-ready.
Tests	None. No test runner, no test script. Biggest gap — the architecture explicitly demands testable business rules (services, state machine, Money value object).
CI config	None. No <code>.github/workflows/</code> .
Env	<code>.env.example</code> exists with <code>DATABASE_URL</code> , JWT secrets, SMTP config, <code>NEXT_PUBLIC_APP_URL</code> . These become CI/CD secrets.
Deploy target	Not chosen yet. Vercel is the least-friction option for Next.js.

Plan: start with the free wins (lint + format + build on every PR), then add tests, then automate deploy. Do not try to build the whole thing at once.

16. Step 1 — Prepare the Repository

Create the workflow folder and confirm the checks run locally first.

```
# from the repo root, D:\Hungry_JU
mkdir -p .github/workflows

# verify every command CI will run actually passes right now
npm ci
npm run lint
npm run format:check
npm run build
```

If `format:check` fails, fix it once with `npm run format` and commit that as its own commit — otherwise the first CI run is red for a reason unrelated to your change.

Also pin the Node version so local and CI agree. Add to `package.json` :

```
"engines": { "node": ">=20.9.0" }
```

And create `.nvmrc` containing `22` .

17. Step 2 — The CI Workflow

Create `.github/workflows/ci.yml`. This is the fast loop: it runs on every PR and every push to `main`.

```

name: CI

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

# a new push to the same branch cancels the previous in-flight run
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: read

jobs:
  quality:
    name: Lint & Format
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - uses: actions/setup-node@v5
        with:
          node-version: 22
          cache: npm          # caches the npm store, keyed on package-lock.json

      - name: Install dependencies
        run: npm ci

      - name: ESLint
        run: npm run lint

      - name: Prettier check
        run: npm run format:check

  build:
    name: Build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - uses: actions/setup-node@v5
        with:
          node-version: 22
          cache: npm

      - name: Install dependencies
        run: npm ci

      # reuse the Next.js compiler cache between runs
      - name: Restore Next.js cache
        uses: actions/cache@v4
        with:
          path: .next/cache
          key: nextjs-${{ hashFiles('package-lock.json') }}-${{ hashFiles('src/**') }}
          restore-keys: |
            nextjs-${{ hashFiles('package-lock.json') }}-

      - name: Build
        run: npm run build

```

```
env:
  # build-time values only. Real secrets are injected at deploy time.
  NEXT_PUBLIC_APP_URL: https://example.com
```

Why two jobs? They run in parallel, so total wall-clock time is the slower one, not the sum. Lint failures also report separately from build failures, which makes the PR page readable at a glance.

Build-time env vars. Next.js inlines `NEXT_PUBLIC_*` into the client bundle at build time, so they must be present during `npm run build` — and must never hold a secret. Server-only variables like `DATABASE_URL` and `JWT_ACCESS_SECRET` are read at runtime; supply them at deploy time, not here. If a module reads them at import time and crashes the build, guard it with a lazy getter rather than putting the real secret into CI.

Commit and push:

```
git switch -c ci/add-github-actions
git add .github/workflows/ci.yml
git commit -m "ci: add lint, format, and build checks"
git push -u origin ci/add-github-actions
```

Open the PR. The **Actions** tab shows the run; the PR page shows the checks.

18. Step 3 — Adding Tests to the Pipeline

The pipeline is only as good as its tests. Vitest is the right fit here — fast, ESM-native, minimal config.

```
npm i -D vitest @vitest/coverage-v8
```

Add to `package.json` scripts:

```
"test": "vitest run",
"test:watch": "vitest",
"test:coverage": "vitest run --coverage"
```

Create `vitest.config.js`:

```
import { defineConfig } from 'vitest/config';

export default defineConfig({
  test: {
    environment: 'node',
    include: ['src/**/*.test.js', 'tests/**/*.test.js'],
    coverage: {
      reporter: ['text', 'lcov'],
      include: ['src/server/**'],
    },
  },
});
```

What to test first in this codebase

Given the layered architecture, priority order:

1. `src/server/services/order-state-machine.js` — pure transition table. Every legal transition returns true, every illegal one throws. Highest value per line of test code.
2. **Money value object** — integer poisha arithmetic, no float drift, immutability.
3. **Business rules BR-01..BR-12 in services** — inject fake repositories through the constructor (the architecture was designed for exactly this).
4. **Zod validators** — valid input passes, malformed input is rejected with the right issue.
5. **Models** — invariants hold; status changes go through methods, not assignment.

Skeleton example:

```
// src/server/services/order-state-machine.test.js
import { describe, it, expect } from 'vitest';
import { OrderStateMachine } from './order-state-machine.js';

describe('OrderStateMachine', () => {
  it('allows placed -> accepted', () => {
    expect(OrderStateMachine.canTransition('placed', 'accepted')).toBe(true);
  });

  it('rejects delivered -> cancelled (terminal state)', () => {
    expect(() => OrderStateMachine.assertTransition('delivered', 'cancelled')).toThrow();
  });

  it('allows cancellation only from placed or accepted (BR-04)', () => {
    expect(OrderStateMachine.canTransition('preparing', 'cancelled')).toBe(false);
  });
});
```

Now add the test job to `ci.yml`:

```
test:
  name: Tests
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v5
    - uses: actions/setup-node@v5
      with:
        node-version: 22
        cache: npm
    - run: npm ci
    - name: Unit tests
      run: npm run test:coverage
    - name: Upload coverage
      if: always() # keep the report even when tests fail
      uses: actions/upload-artifact@v4
      with:
        name: coverage
        path: coverage/
```

Integration tests against a real database

When repositories get implemented, run them against a throwaway Postgres started by the runner:

```

integration:
  name: Integration tests
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:17
      env:
        POSTGRES_USER: test
        POSTGRES_PASSWORD: test
        POSTGRES_DB: hungryju_test
      ports: ['5432:5432']
      options: >-
        --health-cmd pg_isready
        --health-interval 10s
        --health-timeout 5s
        --health-retries 5
  steps:
    - uses: actions/checkout@v5
    - uses: actions/setup-node@v5
      with: { node-version: 22, cache: npm }
    - run: npm ci
    - run: npm run db:migrate          # add this script when migrations exist
      env:
        DATABASE_URL: postgres://test:test@localhost:5432/hungryju_test
    - run: npm run test:integration
      env:
        DATABASE_URL: postgres://test:test@localhost:5432/hungryju_test
        JWT_ACCESS_SECRET: test-access-secret-not-real
        JWT_REFRESH_SECRET: test-refresh-secret-not-real

```

Note the fake secrets: integration tests need *a* value, never the production one.

19. Step 4 — Branch Protection Rules

Once CI is green at least once, make it mandatory. GitHub → **Settings** → **Rules** → **Rulesets** → **New branch ruleset**:

1. Target branch: `main`.
2. Enable **Require a pull request before merging** → 1 approval.
3. Enable **Require status checks to pass** → add `Lint & Format`, `Build`, `Tests` (the `name:` values from the workflow).
4. Enable **Require branches to be up to date before merging**.
5. Enable **Block force pushes**.

From this point, broken code cannot reach `main`.

20. Step 5 — The CD Workflow

Option A — Vercel (recommended for this project)

The lowest-effort path: connect the GitHub repo in the Vercel dashboard. Vercel then gives you, with no YAML at all:

- a preview deployment with its own URL for every pull request;

- a production deployment on every merge to `main` ;
- instant rollback to any previous deployment from the dashboard.

Keep `ci.yml` for the quality gates; let Vercel own deployment. That combination covers most of what a small team needs.

Option B — Explicit deploy workflow (self-hosted / VPS / Docker)

Create `.github/workflows/deploy.yml` :

```

name: Deploy

on:
  push:
    branches: [main]
  workflow_dispatch:      # allows a manual deploy / re-deploy from the UI

concurrency:
  group: deploy-production # never two production deploys at once
  cancel-in-progress: false

permissions:
  contents: read

jobs:
  deploy-staging:
    name: Deploy to staging
    runs-on: ubuntu-latest
    environment:
      name: staging
      url: https://staging.hungryju.example
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v5
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run build
      - env:
          NEXT_PUBLIC_APP_URL: ${{ vars.NEXT_PUBLIC_APP_URL }}

      - name: Deploy
        run: ./scripts/deploy.sh staging
        env:
          DEPLOY_KEY: ${{ secrets.STAGING_DEPLOY_KEY }}

      - name: Smoke test
        run: |
          curl --fail --retry 5 --retry-delay 5 \
            https://staging.hungryju.example/api/health

  deploy-production:
    name: Deploy to production
    needs: deploy-staging      # production only after staging is green
    runs-on: ubuntu-latest
    environment:
      name: production          # attach a required reviewer here in repo settings
      url: https://hungryju.example
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v5
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run build
      - env:
          NEXT_PUBLIC_APP_URL: ${{ vars.NEXT_PUBLIC_APP_URL }}

      - name: Deploy
        run: ./scripts/deploy.sh production
        env:
          DEPLOY_KEY: ${{ secrets.PRODUCTION_DEPLOY_KEY }}

      - name: Health check

```

```
run: |
  curl --fail --retry 5 --retry-delay 5 \
    https://hungryju.example/api/health

- name: Rollback on failure
  if: failure()
  run: ./scripts/rollback.sh production
```

Required approval for production: in GitHub → Settings → Environments → **production**, add yourself as a required reviewer. The job then pauses and waits for a click. That single setting is the difference between continuous delivery and continuous deployment.

Add a health endpoint so the check above has something real to hit — `src/app/api/health/route.js`:

```
export async function GET() {
  return Response.json({ status: 'ok', ts: Date.now() });
}
```

Later, extend it to actually ping the database, so a healthy response means the app can serve requests, not merely that the process started.

21. Step 6 — Database Migrations in CD

Migrations are the most common cause of a broken deploy. Rules:

- **Migrations run before the new code goes live**, as their own step, and must be idempotent and forward-only.
- **Expand → migrate → contract**. Never rename or drop a column in the same release that stops using it:
 1. *Release 1 (expand)*: add the new column, write to both old and new.
 2. *Release 2 (migrate)*: backfill, switch reads to the new column.
 3. *Release 3 (contract)*: drop the old column.

This keeps every intermediate state deployable and rollback-safe — essential for rolling deploys where both versions run simultaneously.

- **Back up before migrating production**. A migration step with no backup step is a one-way door.

```
- name: Backup database
  run: ./scripts/db-backup.sh
  env:
    DATABASE_URL: ${ secrets.DATABASE_URL }

- name: Run migrations
  run: npm run db:migrate
  env:
    DATABASE_URL: ${ secrets.DATABASE_URL }
```

22. Step 7 — Secrets and Environment Variables

Map `.env.example` onto GitHub. **Settings** → **Secrets and variables** → **Actions**:

Key	Where	Note
<code>DATABASE_URL</code>	Secret (per environment)	Different value for staging and production
<code>JWT_ACCESS_SECRET</code>	Secret	Long random string; rotate on any suspected leak
<code>JWT_REFRESH_SECRET</code>	Secret	Must differ from the access secret
<code>SMTP_HOST</code> / <code>SMTP_PORT</code>	Variable	Not sensitive
<code>SMTP_USER</code> / <code>SMTP_PASSWORD</code>	Secret	Mail credentials
<code>MAIL_FROM</code>	Variable	Not sensitive
<code>NEXT_PUBLIC_APP_URL</code>	Variable	Public by definition — it ends up in the client bundle

Use **Environment secrets** (scoped to `staging` / `production`) rather than repository secrets, so a staging job physically cannot read production credentials. Reference them as `${{ secrets.NAME }}` and `${{ vars.NAME }}`.

Confirm `.gitignore` covers `.env*.local` and `.env`. If a real secret ever gets committed, rotating the value is mandatory — rewriting history is not enough, because the old value may already be cloned or cached.

23. Step 8 — Extras Worth Adding

Dependency updates

`.github/dependabot.yml`:

```
version: 2
updates:
  - package-ecosystem: npm
    directory: "/"
    schedule: { interval: weekly }
    open-pull-requests-limit: 5
  - package-ecosystem: github-actions
    directory: "/"
    schedule: { interval: monthly }
```

Each update arrives as a PR that must pass CI — which is exactly why CI is worth building first.

Security scanning

```
security:
  name: Security audit
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v5
    - uses: actions/setup-node@v5
      with: { node-version: 22, cache: npm }
    - run: npm ci
    - run: npm audit --audit-level=high
```

Add GitHub CodeQL (Settings → Code security → enable) for static analysis of the auth and order code — the paths where a logic flaw matters most.

End-to-end tests (later)

Once flows work, add Playwright and run it against a preview deployment: student registers → browses vendor → places order → vendor accepts → status reaches **delivered** . Run on PRs to **main** and nightly, not on every push.

Docker (only if not using Vercel)

```
# next.config.mjs → output: 'standalone' first
FROM node:22-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

FROM node:22-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

FROM node:22-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public ./public
EXPOSE 3000
CMD ["node", "server.js"]
```

24. Rollout Plan and Checklist

Do this in order. Each phase is independently useful; do not start the next until the previous is green.

Phase	Do	Result
1 — Week 1	Add ci.yml with lint + format + build. Fix whatever it finds.	Broken builds and unformatted code can no longer reach a PR unnoticed
2 — Week 1	Turn on branch protection with those checks required	main is always buildable
3 — Week 2	Add Vitest; test the state machine, Money , and validators; add the test job	Business rules are verified on every change
4 — Week 2	Connect Vercel (or write deploy.yml)	Every PR gets a preview URL; merges deploy automatically
5 — Week 3	Add the health endpoint, smoke tests, and staging → production promotion with approval	Deploys are verified, and rollback is one action
6 — Week 4	Integration tests with a Postgres service, migrations in CD, Dependabot, npm audit , CodeQL	Full pipeline

Final checklist

- ☐ `.github/workflows/ci.yml` exists and passes on a PR
- ☐ Every CI command can also be run locally, unchanged
- ☐ `npm ci` used in CI, never `npm install`
- ☐ Node version pinned in the workflow, `.nvmrc`, and `engines`
- ☐ Dependency cache and `.next/cache` configured
- ☐ `concurrency` set so stale runs are cancelled
- ☐ `permissions:` declared, defaulting to `contents: read`
- ☐ Branch protection on `main` requires the CI checks and a review
- ☐ Secrets stored per environment, never in the repo; `.env*` gitignored
- ☐ Test suite exists and runs in CI; coverage uploaded as an artifact
- ☐ Health endpoint exists and the deploy verifies it
- ☐ Production deploy requires approval, or has a tested auto-rollback
- ☐ Migrations run as their own step, backed up first, expand/contract style
- ☐ Dependabot and `npm audit` enabled
- ☐ PR CI finishes in under 10 minutes

25. Troubleshooting Reference

Symptom	Cause and fix
<code>npm ci</code> fails: "lock file does not satisfy package.json"	The lockfile is stale. Run <code>npm install</code> locally and commit the updated <code>package-lock.json</code> .
Build passes locally, fails in CI: "Module not found"	Case-sensitivity. Windows and macOS are case-insensitive; the Ubuntu runner is not. Match import paths to the real filenames exactly.
Build fails on a missing env var	A module reads <code>process.env.X</code> at import time. Either supply a dummy value in the build step, or make the read lazy so it only happens at request time.
Secrets are empty in a PR from a fork	By design — forked PRs get no secrets. Split the workflow so secret-requiring jobs run only on <code>push</code> to <code>main</code> .
Pipeline is very slow	No cache, or jobs chained with <code>needs:</code> that could run in parallel. Add <code>cache: npm</code> , cache <code>.next/cache</code> , and remove unnecessary <code>needs:</code> .
Tests pass alone, fail together	Shared state — a module-level singleton, a shared test database, or a fixed port. Isolate per test, or run serially in a single fork.
Deploy succeeds, site is broken	No verification step. Add a health check plus a smoke test, and a rollback step gated on <code>if: failure()</code> .
Two deploys collide	Missing <code>concurrency</code> group on the deploy workflow.
Required check never appears on the PR	The check name in branch protection must match the job's <code>name:</code> exactly, and the workflow must have run at least once on that branch.

If you do only one thing: add `ci.yml` with lint, format, and build, and make those checks required on `main`. That is roughly 40 lines of YAML, takes 15 minutes, and eliminates the entire category of "someone broke main". Everything else in this document is an extension of that.

Prepared for the Hungry_JU project · Next.js 16 · GitHub Actions